

Frontend Development Guide

HTML • CSS • React • Backend Integration

📄 Complete Code Examples

? 38 Interview Questions

🔗 Coding Problems

📋 AI-Style Interview Questions

🎭 Mock Interview

👁️ Curated with special focus on questions and topics commonly asked to **HCLTech freshers** in technical rounds

Real-world project: React Employee Directory consuming a Spring Boot REST API

Table of Contents

- 1. HTML Fundamentals (Real-World Focus)
- 2. CSS Fundamentals (Flexbox, Grid, Responsive Design)
- 3. JavaScript Essentials Every Fresher Must Know
- 4. React Fundamentals
- 5. Frontend ↔ Backend Integration
- 6. Mini Project: React Employee Directory (Full Code)
- 7. 38 Interview Questions & Answers
- 8. Coding Problems (JS + React)
- 9. AI-Style / Scenario-Based Interview Questions
- 10. Mock Interview Transcript
- 11. HCLTech Fresher Interview — What to Prioritize

1. HTML Fundamentals (Real-World Focus)

1.1 Semantic HTML — Why It's Asked Constantly

Semantic tags describe the meaning of content, not just its appearance. Interviewers use this to check if you write maintainable, accessible markup instead of "div soup".

```
<header>
  <nav>...</nav>
</header>
<main>
  <section>
    <article>
      <h2>Employee Directory</h2>
      <p>Browse and manage company employees.</p>
    </article>
  </section>
  <aside>Filters</aside>
</main>
<footer>© 2026 Company</footer>
```

1.2 Forms — The Real-World Bread and Butter

```
<form id="employeeForm">
  <label for="name">Name</label>
  <input type="text" id="name" name="name" required minlength="2">

  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>

  <label for="salary">Salary</label>
  <input type="number" id="salary" name="salary" min="0" step="1000">

  <button type="submit">Save</button>
</form>
```

Native HTML5 validation attributes (`required`, `minlength`, `type="email"`) prevent obviously bad data from ever reaching JavaScript — always pair this with server-side validation, since client checks can be bypassed.

1.3 Accessibility Basics (ARIA)

```
<button aria-label="Delete employee" onClick={handleDelete}>
  🗑️
</button>


```

Interview tip: mentioning accessibility (alt text, ARIA labels, keyboard navigation) even briefly signals maturity beyond "just making it work".

2. CSS Fundamentals

2.1 Box Model

```
.card {
  width: 300px;
  padding: 16px;
  border: 1px solid #ddd;
  margin: 12px;
  box-sizing: border-box; /* width includes padding + border */
}
```

2.2 Flexbox — Real-World Card Layout

```
.employee-list {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
.employee-card {
  flex: 1 1 260px; /* grow, shrink, basis */
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}
```

2.3 CSS Grid — Dashboard Layout

```
.dashboard {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-rows: 60px 1fr;
  grid-template-areas:
    "sidebar header"
    "sidebar content";
  height: 100vh;
}
.sidebar { grid-area: sidebar; }
.header { grid-area: header; }
.content { grid-area: content; }
```

2.4 Responsive Design with Media Queries

```
.employee-list { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }

@media (max-width: 768px) {
  .employee-list { grid-template-columns: repeat(2, 1fr); }
}
@media (max-width: 480px) {
  .employee-list { grid-template-columns: 1fr; }
}
```

2.5 Specificity Cheat-Sheet

Selector Type	Specificity Weight	Example
Inline style	1000	style="color:red"
ID	100	#header
Class / attribute / pseudo-class	10	.btn, :hover
Element / pseudo-element	1	div, ::before

3. JavaScript Essentials Every Fresher Must Know

React interviews always dip into core JavaScript first — these come up constantly at HCLTech and similar service-based companies.

3.1 var vs let vs const

```
var a = 1;    // function-scoped, hoisted, can be redeclared
let b = 2;    // block-scoped, hoisted but in "temporal dead zone"
const c = 3;  // block-scoped, cannot be reassigned
```

3.2 Closures

```
function makeCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
const counter = makeCounter();
counter(); // 1
counter(); // 2 — "count" is remembered between calls
```

3.3 Promises and async/await

```
function fetchEmployee(id) {
  return fetch(`/api/employees/${id}`)
    .then(res => {
      if (!res.ok) throw new Error("Employee not found");
      return res.json();
    });
}

// async/await version
async function fetchEmployeeAsync(id) {
  const res = await fetch(`/api/employees/${id}`);
  if (!res.ok) throw new Error("Employee not found");
  return res.json();
}
```

3.4 Array Methods Used Constantly in React

```
const names = employees.map(e => e.name);
const highEarners = employees.filter(e => e.salary > 80000);
const totalSalary = employees.reduce((sum, e) => sum + e.salary, 0);
const hasManager = employees.some(e => e.role === "Manager");
const allActive = employees.every(e => e.status === "active");
```

3.5 Destructuring and Spread/Rest

```
const { name, email, ...rest } = employee;
const updatedEmployee = { ...employee, salary: 90000 };
const allEmployees = [...deptA, ...deptB];
```

4. React Fundamentals

4.1 Functional Component with State

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

4.2 useEffect — Data Fetching on Mount

```
import { useState, useEffect } from "react";

function EmployeeList() {
  const [employees, setEmployees] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/employees")
      .then(res => res.json())
      .then(data => setEmployees(data))
      .finally(() => setLoading(false));
  }, []); // empty deps → runs once on mount

  if (loading) return <p>Loading...</p>;
  return (
    <ul>
      {employees.map(e => <li key={e.id}>{e.name}</li>)}
    </ul>
  );
}
```

4.3 Props and Component Composition

```
function EmployeeCard({ employee, onDelete }) {
  return (
    <div className="employee-card">
      <h3>{employee.name}</h3>
      <p>{employee.email}</p>
      <button onClick={() => onDelete(employee.id)}>Delete</button>
    </div>
  );
}
```

4.4 Controlled Forms

```
function EmployeeForm({ onSubmit }) {
  const [form, setForm] = useState({ name: "", email: "", salary: "" });

  const handleChange = (e) =>
    setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = (e) => {
    e.preventDefault();
    onSubmit(form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <input name="salary" value={form.salary} onChange={handleChange} />
      <button type="submit">Save</button>
    </form>
  );
}
```

4.5 Custom Hooks (Reusable Data Fetching)

```

function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    fetch(url)
      .then(res => { if (!res.ok) throw new Error(res.statusText); return res.json(); })
      .then(json => { if (!cancelled) setData(json); })
      .catch(err => { if (!cancelled) setError(err.message); })
      .finally(() => { if (!cancelled) setLoading(false); });
    return () => { cancelled = true; }; // cleanup avoids setting state after unmount
  }, [url]);

  return { data, error, loading };
}

```

4.6 Virtual DOM in One Line

React keeps an in-memory tree of the UI, diffs the new tree against the previous one on every state change, and applies only the minimal set of real DOM updates — this is what makes re-renders fast compared to rebuilding the whole DOM.

5. Frontend ↔ Backend Integration

5.1 Calling a REST API with Axios

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:8080/api",
  headers: { "Content-Type": "application/json" }
});

export const getEmployees = () => api.get("/employees");
export const getEmployee = (id) => api.get(`/employees/${id}`);
export const createEmployee = (data) => api.post("/employees", data);
export const updateEmployee = (id, data) => api.put(`/employees/${id}`, data);
export const deleteEmployee = (id) => api.delete(`/employees/${id}`);
```

5.2 Handling Backend Validation Errors in the UI

Recall the Spring Boot backend returns a structured error like `{ "status": 400, "message": "Validation failed", "fieldErrors": { "email": "Email must be a valid format" } }` on invalid input. The frontend should surface this per-field:

```
async function handleSubmit(form) {
  try {
    await createEmployee(form);
    showToast("Employee created!");
  } catch (err) {
    if (err.response?.status === 400) {
      setFieldErrors(err.response.data.fieldErrors); // show inline under each input
    } else if (err.response?.status === 409) {
      setFormError(err.response.data.message); // e.g. duplicate email
    } else {
      setFormError("Something went wrong. Please try again.");
    }
  }
}
```

5.3 CORS — Why It Bites Every Fresher At Least Once

Browsers block a frontend (e.g. `localhost:3000`) from calling a backend on a different origin (`localhost:8080`) unless the backend explicitly allows it.

```
// Spring Boot side
@CrossOrigin(origins = "http://localhost:3000")
@RestController
@RequestMapping("/api/employees")
public class EmployeeController { ... }
```

5.4 Authentication — Sending a JWT with Every Request

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);
```

5.5 Loading, Error, and Empty States — Real-World UX Checklist

Every screen that fetches data needs to explicitly handle: **loading** (spinner/skeleton), **error** (retry option, readable message), and **empty** (helpful message, e.g. "No employees yet — add your first one"). Interviewers specifically probe for whether you consider all three, not just the happy path.

6. Mini Project: React Employee Directory

A complete, real-world React app that consumes the Spring Boot Employee API (from the backend guide) end-to-end.

6.1 App.jsx — Top-Level State and Routing Logic

```
import { useState, useEffect } from "react";
import { getEmployees, createEmployee, deleteEmployee } from "../api";
import EmployeeForm from "../EmployeeForm";
import EmployeeList from "../EmployeeList";

export default function App() {
  const [employees, setEmployees] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const loadEmployees = () => {
    setLoading(true);
    getEmployees()
      .then((res) => setEmployees(res.data))
      .catch(() => setError("Failed to load employees"))
      .finally(() => setLoading(false));
  };

  useEffect(loadEmployees, []);

  const handleCreate = async (form) => {
    const res = await createEmployee(form); // throws on failure, handled in EmployeeForm
    setEmployees((prev) => [...prev, res.data]);
  };

  const handleDelete = async (id) => {
    await deleteEmployee(id);
    setEmployees((prev) => prev.filter((e) => e.id !== id));
  };

  if (loading) return <p>Loading employees...</p>;
  if (error) return <p role="alert">{error}</p>;

  return (
    <main className="dashboard">
      <h1>Employee Directory</h1>
      <EmployeeForm onSubmit={handleCreate} />
      {employees.length === 0
        ? <p>No employees yet – add your first one above.</p>
        : <EmployeeList employees={employees} onDelete={handleDelete} />}
    </main>
  );
}
```

6.2 EmployeeList.jsx

```
export default function EmployeeList({ employees, onDelete }) {
  return (
    <div className="employee-list">
      {employees.map((emp) => (
        <div className="employee-card" key={emp.id}>
          <h3>{emp.name}</h3>
          <p>{emp.email}</p>
          <p>${emp.salary.toLocaleString()}</p>
          <button onClick={() => onDelete(emp.id)}>Delete</button>
        </div>
      ))}
    </div>
  );
}
```

6.3 EmployeeForm.jsx (with inline field-level errors from backend)

```
import { useState } from "react";

export default function EmployeeForm({ onSubmit }) {
  const [form, setForm] = useState({ name: "", email: "", salary: "", departmentId: "" });
  const [fieldErrors, setFieldErrors] = useState({});
  const [formError, setFormError] = useState("");

  const handleChange = (e) =>
    setForm({ ...form, [e.target.name]: e.target.value });
```

```

const handleSubmit = async (e) => {
  e.preventDefault();
  setFieldErrors({});
  setFormError("");
  try {
    await onSubmit(form);
    setForm({ name: "", email: "", salary: "", departmentId: "" });
  } catch (err) {
    if (err.response?.status === 400) {
      setFieldErrors(err.response.data.fieldErrors || {});
    } else if (err.response?.status === 409) {
      setFormError(err.response.data.message);
    } else {
      setFormError("Something went wrong. Please try again.");
    }
  }
};

return (
  <form onSubmit={handleSubmit}>
    <input name="name" placeholder="Name" value={form.name} onChange={handleChange} />
    {fieldErrors.name && <span className="error">{fieldErrors.name}</span>}

    <input name="email" placeholder="Email" value={form.email} onChange={handleChange} />
    {fieldErrors.email && <span className="error">{fieldErrors.email}</span>}

    <input name="salary" placeholder="Salary" value={form.salary} onChange={handleChange} />
    {fieldErrors.salary && <span className="error">{fieldErrors.salary}</span>}

    {formError && <p className="error">{formError}</p>}
    <button type="submit">Add Employee</button>
  </form>
);
}

```

Notice the pattern: the backend guide's `MethodArgumentNotValidException` handler returns `fieldErrors` as a map — the React form reads that exact shape to show inline errors. This is the frontend-backend contract in action.

7. Interview Questions & Answers (38)

HTML

Q1. What is semantic HTML and why does it matter?

Semantic elements (`<header>` , `<nav>` , `<article>`) describe meaning, improving accessibility (screen readers), SEO, and code readability compared to generic `<div>`s.

Q2. Difference between id and class attributes?

`id` must be unique per page and has higher CSS specificity; `class` can be reused across multiple elements.

Q3. What's the difference between `<script>`, `<script async>`, and `<script defer>`?

Plain `<script>` blocks HTML parsing until it downloads and executes. `async` downloads in parallel and executes as soon as ready (order not guaranteed). `defer` downloads in parallel but executes only after parsing completes, in document order.

Q4. What is the DOM?

The Document Object Model is the browser's in-memory tree representation of the HTML page, which JavaScript can read and modify to change what's rendered.

Q5. What's the difference between GET and POST as used in an HTML form?

GET appends form data to the URL as query parameters (visible, cacheable, limited length); POST sends data in the request body (not visible in the URL, used for creating/mutating data).

CSS

Q6. Explain the CSS box model.

Every element is a box made of content, padding, border, and margin, in that order from inside out. `box-sizing: border-box` makes width/height include padding and border, which is usually more predictable than the default `content-box`.

Q7. Flexbox vs Grid — when do you use each?

Flexbox is for one-dimensional layouts (a row or a column, e.g. a nav bar or card row). Grid is for two-dimensional layouts (rows and columns together, e.g. a dashboard).

Q8. What is CSS specificity and how is it calculated?

It determines which rule wins when multiple rules target the same element, ranked as inline styles > IDs > classes/attributes/pseudo-classes > elements/pseudo-elements, with equal specificity resolved by source order (last wins).

Q9. What is the difference between relative, absolute, fixed, and sticky positioning?

`relative` positions relative to its normal position; `absolute` positions relative to the nearest positioned ancestor; `fixed` positions relative to the viewport and doesn't scroll; `sticky` toggles between relative and fixed depending on scroll position.

Q10. How do you make a layout responsive without a framework?

Use relative units (% , rem , vw/vh), flexible layouts (Flexbox/Grid), and media queries to adjust the layout at defined breakpoints.

JavaScript

Q11. What is hoisting?

JavaScript moves variable and function declarations to the top of their scope during compilation. `var` declarations are hoisted and initialized as `undefined`; `let` / `const` are hoisted but stay in a "temporal dead zone" until their actual declaration line runs.

Q12. What is a closure? Give a practical use case.

A closure is a function that retains access to its outer scope's variables even after the outer function has returned. A practical use is creating private counters or memoized functions.

Q13. Explain the JavaScript event loop.

JavaScript runs on a single thread with a call stack. Asynchronous work (timers, network calls) is handled by the browser/Node APIs, and their callbacks are queued — microtasks (Promises) run before macrotasks (setTimeout) — and the event loop pushes queued callbacks onto the call stack once it's empty.

Q14. Difference between `==` and `===`?

`==` compares values after type coercion; `===` compares both value and type without coercion. `===` is generally preferred to avoid unexpected coercion bugs.

Q15. What is the difference between null and undefined?

`undefined` means a variable has been declared but not assigned a value; `null` is an explicit assignment representing "no value" set intentionally by the programmer.

Q16. Explain map(), filter(), and reduce().

`map()` transforms each element into a new array of the same length. `filter()` keeps only elements matching a condition. `reduce()` folds an array into a single accumulated value.

Q17. What is a Promise and what states can it be in?

A Promise represents an eventual result of an async operation. It can be pending, fulfilled (resolved with a value), or rejected (with an error) — and once settled, it cannot change state again.

Q18. What is debouncing and where would you use it?

Debouncing delays executing a function until a specified time has passed since the last call — commonly used on a search input so an API call only fires after the user stops typing.

React

Q19. What is the Virtual DOM and why does React use it?

It's an in-memory representation of the UI tree. React diffs the new virtual tree against the previous one and applies only the minimal set of real DOM changes, which is faster than re-rendering the whole page directly.

Q20. What is the difference between state and props?

`props` are read-only data passed from a parent to a child component. `state` is data owned and managed within a component itself, and changing it triggers a re-render.

Q21. What does the dependency array in useEffect control?

It determines when the effect re-runs: an empty array runs the effect once on mount; specific values re-run the effect whenever any of those values change; omitting it entirely re-runs on every render.

Q22. Why does React need a unique key prop when rendering lists?

Keys let React's diffing algorithm match list items across renders efficiently and correctly, avoiding incorrect state or DOM reuse when items are reordered, added, or removed.

Q23. What is a controlled component?

An input whose value is driven entirely by React state (via `value` and `onChange`), so React is the single source of truth for the form data, rather than the DOM.

Q24. What are React hooks, and why were they introduced?

Hooks (`useState`, `useEffect`, etc.) let functional components use state and lifecycle-like behavior without writing class components, making logic easier to reuse via custom hooks.

Q25. What is prop drilling and how do you avoid it?

Prop drilling is passing props through several intermediate components that don't need them, just to reach a deeply nested child. It's avoided using the Context API or a state management library.

Q26. What does the Context API solve?

It lets you share data (like a logged-in user or theme) across the component tree without manually passing props at every level.

Q27. What is the difference between useState and useRef?

`useState` updates trigger a re-render. `useRef` stores a mutable value that persists across renders without causing a re-render — often used for DOM references or storing a value you don't want tied to rendering.

Q28. When would you use useMemo or useCallback?

To avoid expensive recalculations or unnecessary re-creation of functions on every render — useful for performance optimization, especially when passing callbacks to memoized child components.

Frontend-Backend Integration

Q29. What is CORS and why does it block API calls during local development?

Cross-Origin Resource Sharing is a browser security mechanism that blocks a page from calling an API on a different origin (protocol/domain/port) unless the server explicitly allows it via response headers.

Q30. How do you handle loading and error states when calling an API from React?

Track separate state variables (e.g. `loading`, `error`, `data`) and conditionally render a spinner, an error message with retry, or the actual data based on their values.

Q31. How does a JWT-based authentication flow typically work between a React app and a backend?

The user logs in via a POST request; the backend returns a signed JWT; the frontend stores it (e.g. in memory or `localStorage`) and attaches it as an `Authorization: Bearer <token>` header on every subsequent request; the backend validates the token on each call.

Q32. Fetch API vs Axios — what's the practical difference?

Fetch is built into the browser but requires manually checking `res.ok` and calling `.json()`; it doesn't reject on HTTP error statuses. Axios automatically parses JSON, rejects on non-2xx responses, and supports interceptors for things like attaching auth headers globally.

Q33. How do you show backend validation errors (like "email invalid") next to the right input field?

Design the backend's error response as a map of field name to message, then in the frontend, read that map and render each message conditionally next to its corresponding input.

Q34. What HTTP status codes should a frontend explicitly handle beyond 200?

At minimum 400 (validation errors), 401 (unauthenticated → redirect to login), 403 (forbidden), 404 (not found), 409 (conflict, e.g. duplicate), and 500 (generic failure message) — each usually needs different UI handling.

Q35. Why should sensitive logic (like access control or price calculation) never live only in the frontend?

Frontend code is fully visible and modifiable by the client (via browser dev tools), so any check done only there can be bypassed — the backend must always re-validate and enforce the actual rule.

Miscellaneous / General Fresher Topics

Q36. What is npm and what does package.json do?

npm is Node's package manager, used to install and manage JavaScript dependencies. `package.json` lists the project's dependencies, scripts, and metadata so the environment can be reproduced consistently.

Q37. What is the difference between a library and a framework?

A library (like React itself, arguably) is called by your code when you need it; a framework dictates the overall structure and calls your code ("inversion of control"), e.g. Angular is more framework-like than React.

Q38. What is responsive web design?

An approach where layout, images, and typography adapt fluidly to different screen sizes and devices using flexible grids, relative units, and media queries, rather than building separate fixed layouts per device.

8. Coding Problems (JavaScript + React)

Problem 1: Remove duplicate objects from an array of employees by email.

```
function dedupeByEmail(employees) {
  const seen = new Set();
  return employees.filter(e => {
    if (seen.has(e.email)) return false;
    seen.add(e.email);
    return true;
  });
}
```

Problem 2: Debounce a search input so the API is called only after the user stops typing for 400ms.

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const debouncedSearch = debounce((term) => searchEmployees(term), 400);
<input onChange={(e) => debouncedSearch(e.target.value)} />
```

Problem 3: Flatten a nested department → employees structure into a single flat array.

```
function flattenDepartments(departments) {
  return departments.reduce(
    (acc, dept) => [...acc, ...dept.employees.map(e => ({ ...e, deptName: dept.name }))]
  );
}
```

Problem 4: Write a custom React hook usePagination that slices a list of employees into pages.

```
function usePagination(items, pageSize = 5) {
  const [page, setPage] = useState(0);
  const totalPages = Math.ceil(items.length / pageSize);
  const currentItems = items.slice(page * pageSize, page * pageSize + pageSize);
  return { currentItems, page, totalPages, nextPage: () => setPage(p => Math.min(p + 1, totalPages - 1)),
    prevPage: () => setPage(p => Math.max(p - 1, 0)) };
}
```

Problem 5: Find employees whose salary is above the department's average salary.

```
function aboveDeptAverage(employees) {
  const totals = {};
  employees.forEach(e => {
    totals[e.deptId] = totals[e.deptId] || { sum: 0, count: 0 };
    totals[e.deptId].sum += e.salary;
    totals[e.deptId].count += 1;
  });
  return employees.filter(e => e.salary > totals[e.deptId].sum / totals[e.deptId].count);
}
```

Problem 6: Implement a simple event emitter (used to understand pub-sub, common in frontend architecture questions).

```
class EventEmitter {
  constructor() { this.listeners = {}; }
  on(event, callback) {
    (this.listeners[event] || []).push(callback);
  }
  emit(event, data) {
    (this.listeners[event] || []).forEach(cb => cb(data));
  }
}
```

Problem 7: Write a React component that retries a failed API call up to 3 times before showing an error.

```
async function fetchWithRetry(url, retries = 3) {
  for (let i = 0; i < retries; i++) {
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error("Request failed");
      return await res.json();
    } catch (err) {
      if (i === retries - 1) throw err;
      await new Promise(r => setTimeout(r, 500 * (i + 1))); // backoff
    }
  }
}
```

9. AI-Style / Scenario-Based Interview Questions

Scenario 1: "Your React app renders a list of 5,000 employees and scrolling feels janky. What would you investigate first?"

Check whether every row is a fully independent re-render on every state change (missing `React.memo` on row components), whether keys are stable (index-based keys on a filtered/sorted list cause unnecessary remounts), and whether the list should be virtualized (rendering only visible rows) rather than rendering all 5,000 DOM nodes at once.

Scenario 2: "A teammate wants to store the JWT in localStorage for simplicity. What tradeoff would you raise?"

localStorage is accessible to any JavaScript running on the page, so it's vulnerable to XSS attacks stealing the token. An httpOnly cookie is safer against XSS (though it introduces CSRF considerations instead) — the right choice depends on the app's threat model, but it's worth explicitly naming the tradeoff rather than defaulting to localStorage silently.

Scenario 3: "The backend team changes an error response's shape from { message } to { error: { code, message } } without telling you. How do you prevent this from breaking your app silently in the future?"

Add a shared API contract (e.g. OpenAPI schema or shared TypeScript types) both teams reference, add defensive parsing on the frontend that fails loudly (not silently) on unexpected shapes, and push for a versioned API or a changelog process so breaking changes are communicated ahead of time.

Scenario 4: "Design the loading UX for a page that needs data from three separate APIs (employee list, department list, current user) before it can render meaningfully."

Fire all three requests in parallel with `Promise.all` (not sequentially, which would be slower), show a single loading state until all resolve, and handle partial failure explicitly — decide whether the page can render with two-out-of-three succeeding or must show a full error, based on which data is essential.

Scenario 5: "How would you structure form validation so both the frontend and backend rules stay in sync over time?"

Keep the backend as the source of truth for validation rules and treat frontend validation purely as a UX layer for instant feedback — never assume the frontend rule alone is sufficient. Where possible, share rule definitions (e.g. a JSON schema or validation config) between both layers so a rule change doesn't require updating two separate, hand-written implementations.

10. Mock Interview Transcript

Interviewer: Walk me through what happens when a user clicks "Add Employee" on your React app, end to end.

Candidate: The form's inputs are controlled, so their values live in component state. On submit, I call `preventDefault()` to stop the native page reload, then call `onSubmit(form)`, which triggers an Axios POST to `/api/employees`. If the backend returns 201, the parent component appends the new employee to its state array, which re-renders the list. If it returns 400, I read the `fieldErrors` map from the response and display each message next to its input. If it's 409, I show a general form-level error like "email already exists".

Interviewer: Why did you choose to keep employees in state at the App level instead of inside EmployeeList?

Candidate: Because both EmployeeForm (adding) and EmployeeList (deleting) need to affect the same data, the state needs to live in their common ancestor — otherwise I'd hit prop drilling or two components with out-of-sync copies of the same list.

Interviewer: What would break if you used the array index as the React key instead of employee.id?

Candidate: If the list is reordered, filtered, or an item is deleted from the middle, index-based keys don't track "which item is which" correctly — React can reuse the wrong DOM node's internal state for a different employee, causing subtle bugs like a text input's typed value jumping to the wrong row.

Interviewer: Suppose the API call takes 3 seconds. What does the user see in your implementation right now, and would you change anything?

Candidate: Right now the form doesn't disable itself, so the user could click submit multiple times, creating duplicate requests. I'd add a `submitting` state that disables the button and shows a spinner until the promise resolves or rejects.

Interviewer: One real bug you might hit integrating this with the Spring Boot backend from scratch?

Candidate: CORS — the browser will block the request from `localhost:3000` to `localhost:8080` until the backend adds `@CrossOrigin` or a global CORS configuration allowing that origin.

11. HCLTech Fresher Interview — What to Prioritize

Based on commonly reported fresher/campus-hire technical rounds at HCLTech and similar IT services companies, these areas tend to carry the most weight:

Priority	Topic	Why It Matters Here
★★★★★	Core JavaScript (closures, hoisting, == vs ===, array methods, promises)	Almost always asked before any React question — treated as a filter round
★★★★★	React basics: props vs state, useState/useEffect, keys in lists, controlled forms	Most fresher projects and assignments are React-based
★★★★☆	HTML/CSS fundamentals: box model, flexbox vs grid, semantic tags, responsive design	Quick screening questions, often asked rapid-fire
★★★★☆	REST API integration: fetch/axios, HTTP status codes, error handling, CORS	Directly tests whether you can actually connect a UI to a real backend
★★★☆☆	Basic OOP concepts (often asked language-agnostically, sometimes in Java)	HCLTech's broader hiring pipeline frequently pairs frontend roles with general programming fundamentals
★★★☆☆	Simple DSA: arrays, strings, basic loops/recursion problems	Common in the aptitude-adjacent coding round before the technical interview
★★☆☆☆	Git basics (clone, commit, push, pull, branch, merge conflicts)	Asked briefly to confirm real project experience

Practical advice: Freshers are rarely expected to know advanced state-management libraries or complex performance optimization. Being able to clearly explain *why* you wrote something a certain way — like why you lifted state up, or why you handled a 400 differently from a 500 — tends to matter more than knowing obscure APIs.

Before the interview: be ready to walk through one of your own projects end-to-end, the way the Mock Interview section above does for the Employee Directory — interviewers consistently ask "walk me through what happens when..." questions to test real understanding over memorized definitions.